NTU AI 2024–HW3 Report

B10902037 Yu Xiang Luo

Details can be found: `GitHub Repository`

Disclaimer: I discussed this homework with *Hao Wei Yei (R13922173)*, so some similarities may exist.

Agents Setup

- For this homework, I modified the original setup by integrating the *Review Analysis Agent* and the *Scoring Agent*. I also utilized regular expressions (regex) to extract all individual scores returned from *Review Analysis and Scoring Agent*, and compute the overall score.
- This optimized setup improves upon the initial design by reducing the likelihood of mis-evaluating from the Scoring Agent, as it no longer needs to reference the full context when assigning scores.

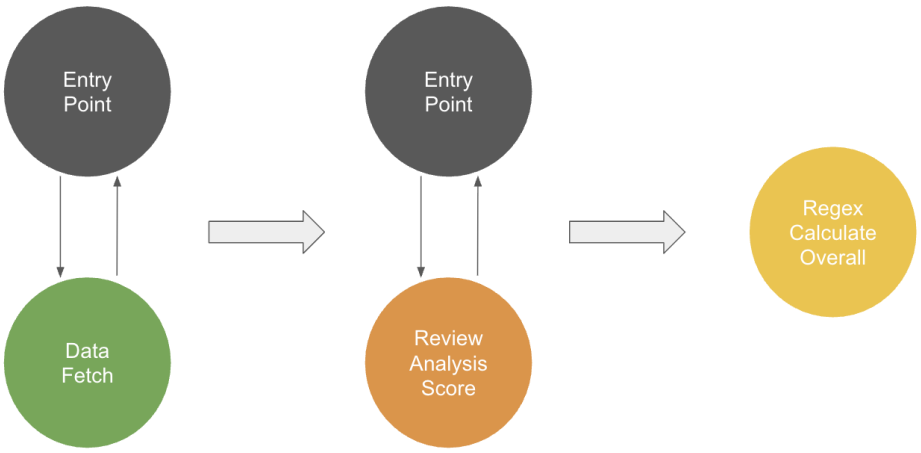

Figure 1: Optimized Agents Workflow

Prompt Design

- I discussed the prompt with *Hao Wei Yei* before passing all public test cases; therefore, our prompts may be similar.
- We originally employed a three-agent setup with detailed, step-by-step instructions. Although this approach retrieved and evaluated scores as expected, it did not demonstrate robustness across multiple rounds.
- Runtime logs showed that, during review analysis, the agent sometimes fetched incomplete reviews, which led to scoring errors.
- To address this issue, we removed the step-by-step output—likely overloading the attention mechanism—and instead evaluated each review immediately after retrieval, so each score depends only on the most recent sentence.
  - **ex**: *The food at McDonald's was average, but the customer service was unpleasant. [food_score: 3, customer_service_score: 2]*

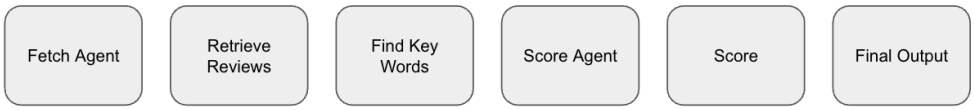

Figure 2: Original three agents workflow

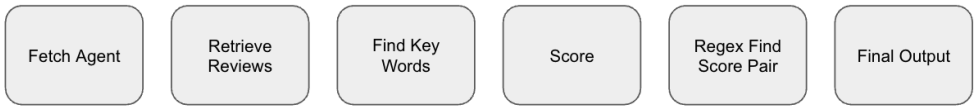

Figure 3: Our two agents workflow

```
ANALYZER = build_agent(
    "review_analyzer_agent",
    '''
    You are the Analyzer Agent responsible for analyzing customer reviews.
    Your task is to extract **one and only one** score pair for **each input review**, with NO EXCEPTIONS.

    ==== SCORING SYSTEM ====
    Each score pair MUST include:
    - a `food_score` (integer from 1-5)
    - a `customer_service_score` (integer from 1-5)

    **Keyword Reference:**
    Score 1: Awful, horrible, disgusting
    Score 2: Bad, unpleasant, offensive
    Score 3: Average, uninspiring, forgettable, meh
    Score 4: Good, enjoyable, satisfying
    Score 5: Awesome, incredible, amazing

    ==== PROCESSING INSTRUCTIONS ====

    - You will receive input in format: {'Restaurant Name': ['Review 1', 'Review 2', ...]}
    - For each review in the list, you repeat it first, then tell the key described word of food and service
    respectively and return the food score and custom service score in such format: [food_score: X, customer_service_score: Y]

    **NOTICE** that meh, uninspiring, and forgettable are neutral descriptions.
    '''
)
```

Figure 4: Our analyze-score agent prompt

Case Analysis: Success and Failure

- To analyze our generations, we define the cases **successful** if the agent repeated the review identically and writed the scores pair in assigned format.

Success Cases

- Example 1: Complete minimal requirements
  - *The food at McDonald's was forgettable, but the customer service was not great. [food_score: 3, customer_service_score: 2]*
- Example 2: Complete minimal requirements and apply easy chain of thoughts based on the instructions
  - *In-n-Out burgers are tasty, always fresh and delicious. The customer service is equally enjoyable, with efficient and friendly staff even during busy times.*
    ** Keyword for food: tasty, Keyword for service: enjoyable*
    ** [food_score: 5, customer_service_score: 4]*

Failure Cases

- Example 1: Incomplete requirements (repeat review)
  - *The food at McDonald's was meh, but nothing special. [food_score: 3, customer_service_score: 2]*

Conclusion and Key Modifications

In conclusion, we first applied a $fetch \rightarrow analyze \rightarrow score$ agent workflow and designed detailed step-by-step instructions to guide the model. This method did not perform well because the lengthy context strained the model's memory. To resolve this, we merged the analysis and scoring steps so the agent evaluates each review immediately after repeating it, thereby easing the memory load—each score now depends only on the repeated review rather than long-term context.

Furthermore, because each score is printed directly after the repeated review, we needed a reliable way to extract the values and compute an overall score. We therefore required the agent to return the score pair in a strict format, which greatly improved evaluation accuracy.

We also noticed that, even with instructions specifying that terms such as *meh*, *uninspiring*, and *forgettable* correspond to a score of 3, the agent often rated them as 2. To mitigate this, we added a ***NOTICE*** line highlighting that these descriptors are neutral and deserve three points.

Lastly, we experimented with a small chain-of-thought structure that prompts the agent to identify key descriptive words before assigning scores. Although the model followed this approach inconsistently, generations that did so tended to achieve a lower MAE.

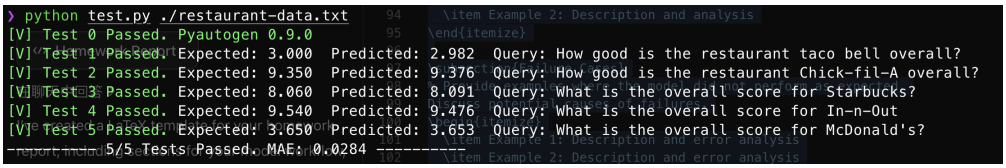

Figure 5: Screentshot of our agent passing all cases

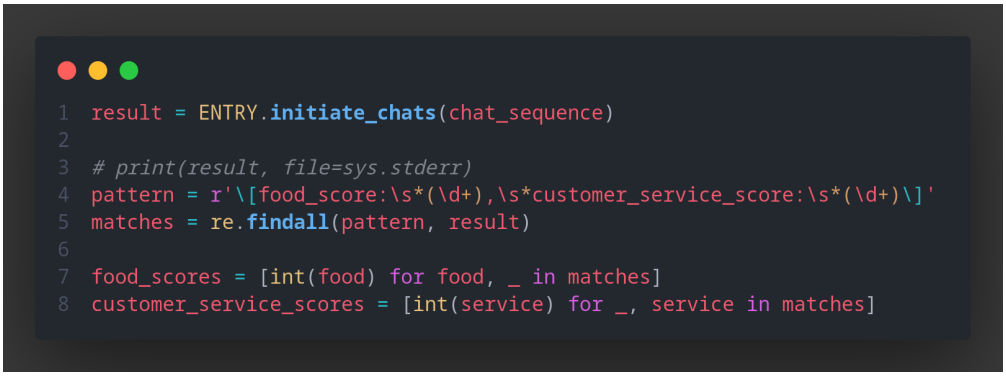

Figure 6: Extraction snippet